Les questions:

1. Quelle est la différence entre un MethodChannel et un EventChannel ? Donnez un exemple de chaque dans votre projet IoT.

Réponse :

MethodChannel : permet à Flutter d'appeler une méthode native Kotlin et de recevoir un résultat

EventChannel : permet à Kotlin d'envoyer continuellement des données à Flutter sous forme de flux (Stream)

2. Pourquoi le Hot Reload ne suffit-il pas après une modification du code Kotlin ?

Réponse :

Le Hot Reload recharge uniquement le code Dart (Flutter). Il ne recompile pas le code natif Kotlin

Après une modification de Kotlin, il faut reconstruire et relancer l'application pour appliquer les changements

3. Que se passe-t-il côté Dart si une branche du when Kotlin oublie d'appeler result.success ? Et si elle l'appelle deux fois ?

Réponse :

Si Kotlin oublie result.success, Flutter attend une réponse qui n'arrive jamais. Le Future reste en attente

Si Kotlin appelle result.success deux fois, cela provoque une erreur, car une réponse ne doit être envoyée qu'une seule fois pour un appel

4. À l'étape 4, pourquoi lit-on le micro dans Dispatchers.IO, puis envoie-t-on la valeur dans Dispatchers.Main ?

Réponse :

Dispatchers.IO permet d'effectuer les opérations de lecture du microphone sans bloquer l'interface

Dispatchers.Main permet d'envoyer les événements vers Flutter depuis le thread principal

5. Quel est le rôle de onCancel ? Que se passe-t-il si on n'y libère pas le capteur ou le micro ?

Réponse :

onCancel est appelée lorsque Flutter arrête d'écouter un EventChannel.

Elle permet de libérer les ressources :

Arrêter l'écoute du capteur

Arrêter le microphone

Annuler la Coroutine si nécessaire

6. Pourquoi MainActivity garde-t-elle resultatPermission au lieu de répondre tout de suite à Dart ?

Réponse :

Parce que la demande de permission Android est asynchrone

MainActivity doit attendre la réponse de l'utilisateur (autoriser ou refuser le microphone). Elle conserve donc resultatPermission pour répondre à Flutter après le résultat de la demande

7. Le plugin battery_plus donne la batterie sans Kotlin. Expliquez pourquoi c'est faux.

Réponse :

C'est faux parce que battery_plus utilise lui aussi du code natif Android pour accéder aux informations de la batterie.

La différence est que le développeur n'a pas besoin d'écrire lui-même le code Kotlin : le plugin le fournit déjà et communique avec Flutter à travers les mécanismes de plateforme